

Unit 3 · Topic Notes

Role-Based Access Control (RBAC) and Protected Routes

Course: Full Stack Application Development with MERN

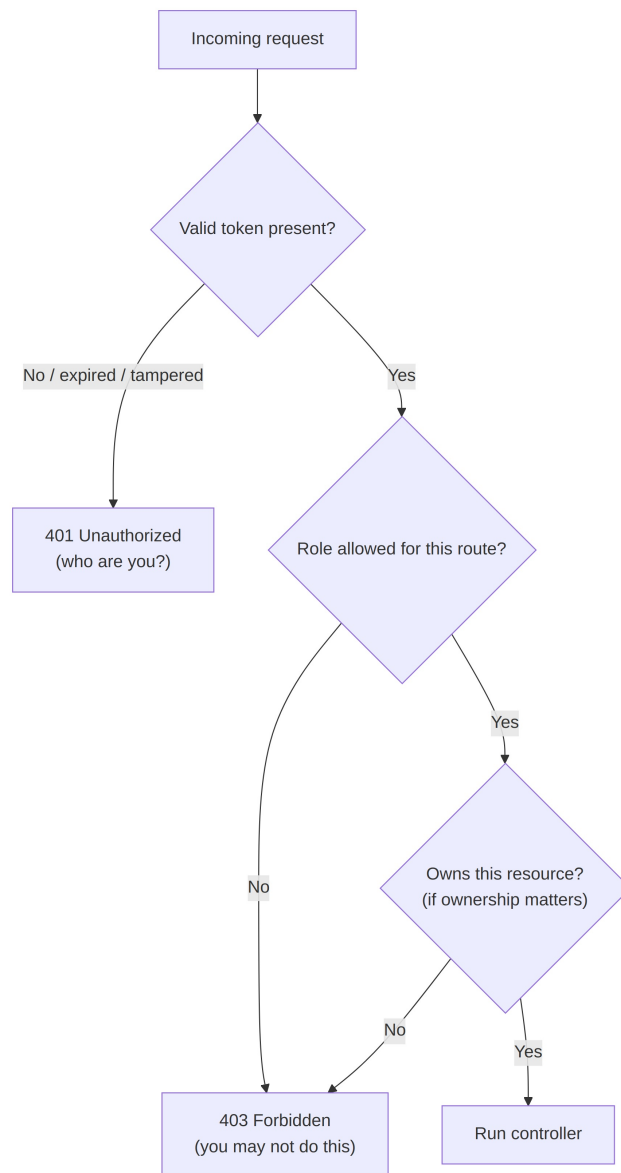
Learning Objectives

1. Restrict API routes by role with an `authorize` middleware
 2. Enforce ownership so users only touch their own data
 3. Protect React routes and show role-specific UI
 4. Explain why frontend guards are not security
 5. Handle the edge cases: stale roles, missing tokens, self-demotion, redirect loops
-

1. Three Layers of Access Control

Layer	Question	Example
Authentication	Are you logged in?	<code>protect</code> middleware
Role check (RBAC)	Does your role allow this kind of action?	Only <code>admin</code> can delete a product
Ownership check	Is this specific record yours?	A user can read only their own orders

Most real bugs happen in the third layer, because it is easy to forget.



Access decision flow

2. Roles on the Server

The `role` field already exists on the `User` model (`"user"` or `"admin"`). Now add the guard:

```
// middleware/authorize.js
export const authorize = (...allowedRoles) => (req, res, next) => {
  if (!req.user) {
    return res.status(401).json({ message: "Not authenticated" }); // protect was forgotten or failed
  }
  if (!allowedRoles.includes(req.user.role)) {
    return res.status(403).json({ message: "You do not have permission to do this" });
  }
  next();
};
```

Use it **after** `protect`, because `protect` is what sets `req.user`:

```
// routes/productRoutes.js
router.get("/", getProducts); // public
router.post("/", protect, authorize("admin"), createProduct); // admin only

router
  .route("/:id")
  .all(validateId)
  .get(getProduct) // public
  .patch(protect, authorize("admin"), updateProduct)
  .delete(protect, authorize("admin"), deleteProduct);
```

```
// routes/userRoutes.js (admin area)
router.use(protect, authorize("admin")); // applies to every route below
router.get("/", listUsers);
router.patch("/:id/role", updateUserRole);
```

2.1 More than two roles

```
// User schema: role: { type: String, enum: ["user", "editor", "admin"], default: "user" }
router.post("/articles", protect, authorize("editor", "admin"), createArticle);
router.delete("/articles/:id", protect, authorize("admin"), deleteArticle);
```

2.2 Permission map (scales better than many role names)

```
// config/permissions.js
export const PERMISSIONS = {
  user: ["product:read", "order:create", "order:read:own"],
  editor: ["product:read", "product:create", "product:update"],
  admin: ["*"],
};

export const can = (permission) => (req, res, next) => {
  const perms = PERMISSIONS[req.user?.role] ?? []; // unknown role = no permissions
  if (perms.includes("*") || perms.includes(permission)) return next();
  res.status(403).json({ message: "Forbidden" });
};

// router.delete("/:id", protect, can("product:delete"), deleteProduct);
```

3. Ownership Checks (Preventing IDOR)

IDOR (Insecure Direct Object Reference) means a logged-in user changes an id in the URL and reads someone else's data.

```
// VULNERABLE: any logged-in user can read ANY order by guessing its id
export const getOrder = async (req, res) => {
  const order = await Order.findById(req.params.id);
  res.json(order);
};
```

```
// SAFE: the query itself includes the owner
export const getOrder = async (req, res) => {
  const filter = { _id: req.params.id };
  if (req.user.role !== "admin") filter.user = req.user._id; // admins see everything

  const order = await Order.findOne(filter);
  if (!order) return res.status(404).json({ message: "Order not found" });
  res.json(order);
};

export const createOrder = async (req, res) => {
  const order = await Order.create({
    user: req.user._id, // from the verified token, NEVER from req.body.user
    items: req.body.items,
    total: computeTotal(req.body.items), // recompute on the server, never trust a client total
  });
  res.status(201).json(order);
};
```

Design choice: for someone else's order the code returns `404`, not `403`. A `403` would confirm that the order exists. For private data, `404` leaks less.

Listing: always scope list queries too:

```
export const myOrders = (req, res) => Order.find({ user: req.user._id }).sort({ createdAt: -1 }).then((o) => res.json(o));
```

4. Managing Roles Safely

```
// controllers/userController.js
export const updateUserRole = async (req, res) => {
  const { role } = req.body ?? {};
  if (!["user", "editor", "admin"].includes(role)) {
    return res.status(400).json({ message: "Invalid role" });
  }
  if (req.params.id === req.user.id) {
    return res.status(400).json({ message: "You cannot change your own role" });
  }
  const user = await User.findByIdAndUpdate(req.params.id, { role }, { new: true, runValidators: true });
  if (!user) return res.status(404).json({ message: "User not found" });
  res.json({ id: user._id, email: user.email, role: user.role });
};
```

How does the first admin exist? Not through the public register API (which ignores `role`). Use a one-off script:

```
// scripts/seedAdmin.js -> node scripts/seedAdmin.js
import "dotenv/config";
import mongoose from "mongoose";
import User from "../models/User.js";

await mongoose.connect(process.env.MONGO_URI);
const exists = await User.findOne({ email: process.env.ADMIN_EMAIL });
if (!exists) {
  await User.create({ name: "Admin", email: process.env.ADMIN_EMAIL, password: process.env.ADMIN_PASSWORD, role: "admin" });
  console.log("Admin created");
}
await mongoose.disconnect();
```

5. Protected Routes in React

5.1 Auth context that knows when it is still loading

```
// src/context/AuthContext.jsx
import { createContext, useContext, useEffect, useState } from "react";
import api from "../services/api";

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true); // true until we KNOW who the user is

  useEffect(() => {
    const token = localStorage.getItem("token");
    if (!token) return setLoading(false);
    api.get("/auth/me")
      .then((res) => setUser(res.data.user))
      .catch(() => localStorage.removeItem("token"))
      .finally(() => setLoading(false));
  }, []);

  const login = async (email, password) => {
    const { data } = await api.post("/auth/login", { email, password });
    localStorage.setItem("token", data.token);
    setUser(data.user);
  };

  const logout = () => { localStorage.removeItem("token"); setUser(null); };

  return (
    <AuthContext.Provider value={{ user, loading, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

export const useAuth = () => useContext(AuthContext);
```

5.2 The route guard

```
// src/components/ProtectedRoute.jsx
import { Navigate, Outlet, useLocation } from "react-router-dom";
import { useAuth } from "../context/AuthContext";

export default function ProtectedRoute({ roles }) {
  const { user, loading } = useAuth();
  const location = useLocation();

  if (loading) return <p>Loading...</p>; // do NOT redirect yet
  if (!user) return <Navigate to="/login" replace state={{ from: location }} />;
  if (roles && !roles.includes(user.role)) return <Navigate to="/unauthorized" replace />;

  return <Outlet />;
}
```

5.3 Using it in the route table

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/login" element={<Login />} />
  <Route path="/unauthorized" element={<Unauthorized />} />

  <Route element={<ProtectedRoute />} /* any logged-in user */
    <Route path="/dashboard" element={<Dashboard />} />
    <Route path="/orders" element={<MyOrders />} />
  </Route>

  <Route element={<ProtectedRoute roles={["admin"]} />} /* admins only */
    <Route path="/admin" element={<AdminPanel />} />
  </Route>
</Routes>
```

5.4 Send the user back where they were going

```
// Login.jsx
const navigate = useNavigate();
const location = useLocation();
const from = location.state?.from?.pathname || "/dashboard";

const handleSubmit = async (e) => {
  e.preventDefault();
  try {
    await login(email, password);
    navigate(from, { replace: true });
  } catch (err) {
    setError(err.response?.data?.message || "Login failed");
  }
};
```

5.5 Role-based UI

```
const { user } = useAuth();
return (
  <nav>
    <Link to="/">Home</Link>
    {user && <Link to="/orders">My Orders</Link>}
    {user?.role === "admin" && <Link to="/admin">Admin</Link>}
  </nav>
);
```

The most important rule of this topic: hiding a button or blocking a route in React is **user experience, not security**. Anyone can open DevTools, edit JavaScript, or call your API directly with Postman. **Every rule must be enforced again on the server.**

6. Edge Cases

- 1. protect before authorize, always.** `router.delete("/:id", authorize("admin"), protect, ...)` runs `authorize` while `req.user` is still `undefined`. The guard above returns `401` instead of crashing, but the order is still a bug.
- 2. Default deny.** New routes should be locked unless you deliberately open them. An unknown role should get *no* permissions (see `PERMISSIONS[role] ?? []`).
- 3. Stale role in the frontend.** If an admin demotes a user, that user's open browser tab still shows admin links until the next `/auth/me`. The API still blocks them (because the server reads the role from the database), which is why server enforcement matters. Re-fetch `/auth/me` on focus or after sensitive actions.
- 4. Token expires while the user is on a protected page.** The next API call returns `401`; the Axios interceptor clears the token and redirects to `/login`. Verify this path, don't assume it.
- 5. Refresh on `/dashboard`.** React state is empty at page load. Without the `loading` flag the guard sees `user === null` for one render and bounces a logged-in user to `/login`. Always wait for the initial `/auth/me` check.
- 6. role inside the JWT goes stale.** If the token says `"role": "admin"` for 15 minutes after demotion, the user keeps admin power. Reading the role from the database on each request (as `protect` does) avoids this. The cost is one query per request; cache carefully if you optimise.
- 7. Open redirect.** If you support `/login?next=/somewhere`, validate it: it must start with a single `/` and not `//` or `http`. Otherwise `?next=//evil.com` sends users to an attacker after login. (Using router `state.from` as above avoids the problem entirely.)
- 8. 401 vs 403 in the UI.** `401` → send to login. `403` → show "You don't have access". Redirecting a `403` to login traps a logged-in user in a loop.
- 9. Mass assignment of role.** `User.create(req.body)` or `findByIdAndUpdate(id, req.body)` on profile updates lets any user send `{"role": "admin"}`. Whitelist fields (`name`, `phone`), never spread the body.
- 10. Admins deleting or demoting themselves.** The last admin locks everyone out. Block self-demotion (done

above) and self-deletion, and consider a “cannot remove the final admin” count check.

11. Logging out in one tab. Other tabs keep working with their in-memory `user`. Listen for the storage event:

```
useEffect(() => {
  const onStorage = (e) => { if (e.key === "token" && !e.newValue) setUser(null); };
  window.addEventListener("storage", onStorage);
  return () => window.removeEventListener("storage", onStorage);
}, []);
```

12. Test with the wrong user. For every protected endpoint, test as: no token, a normal user, the owner, another normal user, an admin. Most authorization bugs appear in the “another normal user” case.

7. Hands-On Lab (60 min, pairs)

1. Add `authorize` and protect the product routes so that only admins can create, edit and delete.
2. Create two normal users and one admin (via the seed script). Using Postman, fill in this table with the actual status codes:

Request	No token	User A	Admin
<code>POST /api/products</code>			
<code>DELETE /api/products/:id</code>			
<code>GET /api/orders/:id</code> (owned by A)			
<code>GET /api/orders/:id</code> (owned by B), as A			

3. Implement the ownership-scoped `getOrder` and prove user A cannot read user B’s order.
4. Build `AuthContext` and `ProtectedRoute` in React. Verify: refreshing `/dashboard` while logged in stays on the dashboard; visiting `/admin` as a normal user shows “Unauthorized”.
5. Open DevTools, edit the stored user object to `role: "admin"`, and try to use an admin feature. What stops you? (The answer is the point of the lab.)

8. Quick Assessment

1. When do you return `401` and when `403`?
2. What is an IDOR bug? Show a one-line fix for `Order.findById(req.params.id)`.
3. Why must the server enforce roles even when the React app already hides the admin button?
4. Why does `ProtectedRoute` check a `loading` flag?
5. Why should `createOrder` take `user` from `req.user` instead of `req.body`?
6. A user is demoted from admin while logged in. Where in the system does the change take effect immediately, and where is it delayed?

9. Common Mistakes to Avoid

- Checking roles only in React
- Trusting ids, totals or roles sent by the client
- Forgetting the ownership filter on list endpoints
- Putting `authorize` before `protect`
- Letting the public register endpoint accept a `role`